ShadowMem Backend Audit & Upgrade Roadmap

System Architecture, Security Findings, and Incremental Modernization Plan

Target Codebase: C:\Users\Himesh\Desktop\ShadowMem | **Audit Date:** October 05, 2026 - 18:26 | **Scope:** Backend Services, Core Engine, ML Inference & API Layer

---

1. Executive Summary & Assessment Scorecard

ShadowMem provides an impressive, technically sound foundation for volatile Windows memory forensics, leveraging user-mode Win32 memory introspection (VirtualQueryEx, NtQueryInformationThread) paired with a 7-feature machine learning classification pipeline. However, the current backend implementation exhibits significant architectural debt, tight coupling, and security vulnerabilities that inhibit enterprise reliability and scalability. Most critically, the codebase is anchored around a monolithic 3,434-line app.py (including ~2,478 lines of inline HTML/JS), an unresolved dual-server conflict (server.py vs. app.py), unauthenticated process-manipulation endpoints, and zero automated test scaffolding for core Win32 subsystems.

Subsystem / Domain	Health Rating	Key Finding / Root Limitation	Target State
Architecture & Modularity	**CRITICAL**	3,434-line monolithic app.py with 2,478 lines of embedded HTML; duplicated Win32 logic across 4 files.	Clean 4-tier layer: Routers → Services → Models → Core Engine.
Server Architecture (ERR-09)	**HIGH RISK**	server.py and app.py both bind port 8000; dead WebSocket loop in app.py; divergent payloads.	Single authoritative FastAPI server with live alert broadcaster; server.py fully retired.
API Design & Typing	**HIGH RISK**	No API versioning; no Pydantic request/response models; POST /api/isolate uses query param for reason.	Standardized /api/v1/ REST API with strict Pydantic schemas and OpenAPI documentation.
Security & Containment	**CRITICAL**	Unauthenticated /api/isolate allows arbitrary thread suspension; permissive CORS (* + credentials).	Local token/API-key authorization, strict CORS whitelist, loopback-only binding.
Configuration & Secrets	**MODERATE**	Hardcoded ports, file paths, JIT allowlists, and ML thresholds scattered throughout Python files.	Centralized config.py leveraging pydantic-settings and .env file support.
Performance & Concurrency	**HIGH RISK**	Linear process scan loops block async event loop for 2-10+ seconds; global mutable cache race conditions.	Offloaded CPU-bound scans via asyncio.to_thread or background worker queue.
Test Scaffolding & CI	**CRITICAL**	Zero unit test framework; tests are standalone scripts requiring live Windows admin processes.	Pytest suite with mocked ctypes/Win32 APIs runnable on any platform and CI/CD pipelines.

2. Detailed Codebase Mapping & Component Roles

The ShadowMem backend currently consists of 18 Python scripts in the root directory alongside the Next.js frontend. The functional responsibilities of key modules are cataloged below:

Module / File	Lines / Size	Actual Functional Responsibility	Architectural Flaws / Issues Identified
`app.py`	3,434 lines 147.3 KB	Main FastAPI application. Serves Next.js static files and fallback HTML; exposes /api/scan, /api/isolate/{pid}, /api/resume/{pid}, /api/audit, and WebSocket alerts.	Massive monolith: embeds 2,478 lines of raw HTML; duplicates Win32 declarations; stores unpersisted mutable global state (all_processes); dead WebSocket loop.
`server.py`	290 lines 10.9 KB	Legacy/mock FastAPI server built for early hackathon demos. Emits synthetic telemetry heartbeats, randomized alerts, and mock memory maps over /ws/alerts.	**ERR-09 root cause:** Hardcodes port 8000; lacks memory scanning and isolation; divergent schema on /simulate/alert compared to app.py.
`memoryscanner.py`	185 lines 6.8 KB	Low-level memory forensics engine using ctypes (kernel32, ntdll) and psutil. Performs linear VAD region walk, detects RWX pages, checks RW→RX flips, and extracts thread start addresses.	Synchronous blocking execution; incomplete 64-bit address space ceiling check; no handle leak recovery on unexpected ctypes exceptions; lacks abstraction for testing.
`detecter.py`	136 lines 5.0 KB	Command-line forensic scanning tool. Imports memoryscanner, loads pickled ML model, extracts 7 features, applies JIT suppression, and prints terminal summary.	Redundant copy of JIT allowlist logic and feature vector assembly that also exists verbatim inside app.py.

`train_model.py`	615 lines 30.1 KB	Advanced ML training pipeline. Generates 6,000 synthetic forensic vectors across 9 subclasses (5 benign, 4 malicious), fits HistGradientBoostingClassifier with RobustScaler, verifies canary profiles, and dumps pickle.	Produces an unvalidated Python pickle artifact (code execution risk); synthetic distributions have hardcoded parameters not derived from real telemetry baselines.
`technique_docs.py`	154 lines 7.5 KB	Static MITRE ATT&CK; reference data. Maps multi-class indices (5-8) to display names, MITRE IDs (T1055.001, T1055.012, T1620, T1055.003), indicators, and response playbooks.	Coupled tightly to index order in train_model.py; hardcoded dictionaries without schema validation or internationalization.
`malware_simulator.py`	498 lines 19.8 KB	Safe user-mode attack harness. Allocates RWX buffers with NOP sleds, spawns worker threads, performs VirtualProtect flips, and registers threats with the prototype.	Documentation instructs users to launch server.py on port 8000, perpetuating developer confusion and port collisions.
`test_master_suite.py` `test_sandbox.py`	270 lines 275 lines	Validation suites executing simulated attacks, live server scans, process isolation, and PDF generation.	Not integrated into standard testing runners (e.g. pytest); requires admin privileges and live running servers; non-reproducible in CI.

3. In-Depth Technical Audits & Vulnerability Analysis

A. Deep Dive: The ERR-09 Dual-Server Conflict (Current Reality)

Historical Context: ShadowMem originally emerged with two disjoint backends: server.py was designed as a lightweight WebSocket mock streaming synthetic telemetry for early UI design, while app.py was created as the authoritative forensics engine. A previous bug ticket (ERR-09) documented that running both servers produced port binding collisions on port 8000.

Audit Verification — Does ERR-09 Still Exist?
YES. While partially mitigated by migrating route declarations into app.py, the conflict remains fundamentally active:

1. **Port Collision Hazard:** Both app.py and server.py retain uvicorn.run(app, host='0.0.0.0', port=8000) at their entry points. Any developer or script running server.py while launch_shadowmem.bat is active triggers an immediate WinError 10048 (WSAEADDRINUSE).
2. **Dead WebSocket Loop in app.py:** In app.py, /ws/alerts was added, but it only sends a single static welcome JSON and then executes while True: await asyncio.sleep(60). Live scan results from /api/scan are **never broadcast** to connected WebSocket clients. In contrast, server.py actually streams continuous heartbeats and memory maps.
3. **Divergent API Signatures on /simulate/alert:** In server.py, POST /simulate/alert expects a JSON body validated by Pydantic (SimulateAlertRequest(scenario, severity)). In app.py, line 875 defines async def simulate_alert(): with **no parameters**, hardcoding Cobalt Strike Beacon strings and ignoring caller parameters.
4. **Stale Documentation and Script References:** malware_simulator.py (line 38) explicitly tells the operator to start server.py for WebSocket alerts. Meanwhile, restart_server.py relies on aggressive taskkill /F /PID commands against port 8000 to recover from deadlocks.

B. Win32 & psutil Memory Telemetry Flow (Collection → Inference → Response)

The collection pipeline in memoryscanner.py operates as follows:

1. **Process Discovery:** scan_all_processes() iterates across all active processes via psutil.process_iter(['pid']). For each PID, it acquires basic metadata (process name, exe path, thread count).
2. **Process Handle Acquisition:** Invokes kernel32.OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, False, pid). Protected system processes (e.g. LSASS, CSRSS) fail gracefully with Access Denied.
3. **Linear VAD Region Walking:** Starting at virtual address 0x0, the engine iteratively calls kernel32.VirtualQueryEx(). For every committed private region (MEM_COMMIT & MEM_PRIVATE), it checks for executable permissions (PAGE_EXECUTE_READ, PAGE_EXECUTE_READWRITE). If found, unbacked_exec_count increments. If RWX, rwx_count and total byte sizes increment.
4. **VAD Protection Anomaly Detection:** Flags allocations whose current protection is executable but whose initial allocation was non-executable data (AllocationProtect == PAGE_READWRITE). This is the hallmark signature of runtime payload staging and VirtualProtect flips.
5. **Thread Start Address Resolution:** Queries active threads using undocumented ntdll.NtQueryInformationThread(ThreadQuerySetWin32StartAddress). If a thread's start address falls inside an unbacked private executable region, thread_unbacked_start increments (indicating shellcode/remote execution).
6. **ML Inference & Serialization:** Telemetry is passed to evaluate_process() in app.py, which runs the 7-feature vector through model.predict_proba(), evaluates JIT allowlists, and aggregates JSON results.

C. Security Vulnerabilities, Concurrency & API Architecture

• **Severe Security Hole in Process Containment:** POST /api/isolate/{pid} and POST /api/resume/{pid} accept arbitrary PIDs and suspend process threads on the host Windows system. These endpoints have **no authentication**, **no authorization**, and **no CSRF tokens**. Because app.py binds 0.0.0.0, any attacker on the local network or any malicious website running in the analyst's browser (via DNS rebinding or CSRF) can suspend arbitrary user processes.

• **CORS Specification Violation:** In app.py lines 252-256, CORS is configured with allow_origins=['*'] and allow_credentials=True. Under W3C CORS specifications, browsers reject requests where wildcard origins are paired with credentialed requests.

• **Event Loop Blocking (Async Starvation):** api_scan() is defined as a synchronous route that invokes memoryscanner.scan_all_processes(). This function executes a synchronous, CPU-intensive loop querying hundreds of processes and thousands of memory regions via Win32 syscalls. During this 2-10 second execution, concurrent API calls or WebSocket frames can suffer severe latency spikes.

• **Unsafe State Mutation & Global Leakage:** app.py uses global in-memory variables (all_processes: list and _isolation_audit: list). Concurrent requests create race conditions, and all forensic state and audit trails are lost when the Uvicorn process restarts.

• **Insecure ML Model Loading:** shadowmem_model.pkl is deserialized using Python's standard pickle.load() without signature verification. If a malicious model file is substituted, arbitrary code execution occurs inside the backend supervisor.

4. Clean Integration Points for Upcoming Capabilities

ShadowMem has two planned major extensions: a **Docker-based dynamic analysis sandbox (via WSL2)** and a **Supply Chain Security Scanner**. Currently, there is no modular architecture to support them without severely polluting app.py. The recommended integration architecture is detailed below:

Planned Extension	Current Codebase Readiness	Recommended Integration Architecture	Interface / Contract Design
Docker Sandbox (via WSL2) **Safe detonator for unknown binaries and memory dump analysis.**	**Zero Readiness** All scanning is hardcoded to the local Windows host address space.	Create backend/services/sandbox_service.py communicating with Docker daemon via WSL2 socket or Docker SDK. Isolates detonated processes in lightweight Linux/Windows containers, dumps memory pages, and streams execution artifacts back to the main engine.	`BaseSandboxProvider:` • `detonate_sample(file_bytes)` • `capture_memory_snapshot()` • `stream_sandbox_logs()` • `terminate_sandbox()`
Supply Chain Scanner **Analyzes dependencies, requirements, lockfiles, and imports for malicious packages.**	**Zero Readiness** No concept of static package analysis, lockfile parsing, or vulnerability databases.	Create backend/services/supply_chain_service.py. Parses requirements.txt, package.json, and PE import tables (via pefile). Queries OSV/PyPI advisory APIs and checks package hashes against known malicious packages.	`SupplyChainScanner:` • `scan_python_manifest()` • `scan_npm_manifest()` • `inspect_pe_imports(file_path)` • `check_advisories(pkg_spec)`

5. Prioritized Backend Upgrade Recommendations

To ensure incremental delivery without breaking existing frontend interfaces or demo flows, the modernization plan is divided into four sequential phases. Each phase can be implemented and validated independently.

Phase / Step	Upgrade Recommendation	What It Fixes / Value Delivered	Effort	Risk
PHASE 1.1 **Immediate**	**Permanently Resolve ERR-09 & Retire server.py** Migrate live alert broadcasting to app.py; wire real scan detections into WebSockets; archive or repurpose server.py into an explicit test mock in tests/mocks/.	Eliminates port 8000 collisions permanently; fixes dead WebSocket loop; ensures frontend receives real alerts during live scans.	Small	Low
PHASE 1.2 **Immediate**	**Introduce Pydantic Schemas & Versioned REST Routes** Define request/response schemas (ProcessScanResponse, IsolateRequest, AuditResponse, WebSocketAlertSchema); mount under /api/v1/ while maintaining temporary backward-compatibility aliases.	Provides runtime input validation, eliminates ad-hoc JSON errors, generates automatic OpenAPI/Swagger docs, and stabilizes frontend contracts.	Medium	Low

PHASE 1.3 Immediate	**Centralized Settings & Configuration (config.py)** Implement Settings(BaseSettings) via pydantic-settings. Manage port, host, paths, thresholds (70/35), JIT allowlists, and CORS origins via environment variables and .env file.	Eliminates hardcoded ports and thresholds scattered across 5+ files; enables seamless multi-environment deployment.	Small	Very Low
PHASE 2.1 Refactor	**Modular Package Layering (De-monolith app.py)** Split backend into standard layers: api/v1/endpoints/ (routers), services/ (business logic), core/ (Win32 ctypes engine & ML), and models/ (schemas).	Reduces app.py from 3,434 lines to a clean ~60-line bootstrap script; ends duplicated Win32 and JIT logic.	Medium	Low
PHASE 2.2 Refactor	**Decouple Embedded HTML Fallback** Extract INDEX_HTML (2,478 lines) out of Python code into a dedicated static template directory (templates/fallback.html). Mount cleanly via FastAPI static/template engine.	Restores Python file cleanliness, improves IDE syntax highlighting and linting, reduces backend memory overhead.	Small	Very Low
PHASE 2.3 Quality	**Structured JSON Logging & Standardized Error Handling** Implement centralized exception handlers (HTTPException, Win32Error, ValidationError) and structured logging with timestamp, correlation ID, and level.	Guarantees clean, demo-safe error responses; eliminates raw Python stack trace leaks to the client; simplifies log analysis.	Medium	Low
PHASE 2.4 Testing	**Pytest Scaffolding & Win32 Mocking** Establish tests/ with conftest.py, FastAPI TestClient, and mock fixtures for kernel32.VirtualQueryEx and ntdll.NtQueryInformationThread.	Enables automated regression testing in Linux CI/CD environments without requiring live Windows admin credentials.	Medium	Low
PHASE 3.1 Security	**API Security Hardening & Process Isolation Controls** Bind server to 127.0.0.1 by default; correct CORS to explicit origins; require local bearer token or session header for /api/v1/isolate.	Prevents unauthorized remote network clients and malicious web pages from weaponizing ShadowMem to suspend Windows processes.	Small	Medium
PHASE 3.2 Performance	**Non-Blocking Async Memory Scanning** Wrap CPU-bound scan_all_processes() in asyncio.to_thread() or background worker pool. Add scan progress events over WebSocket.	Prevents event loop starvation during full system scans; delivers responsive UI feedback with real-time scan progress percentages.	Medium	Low
PHASE 3.3 Reliability	**Lightweight SQLite Persistence Layer** Replace ephemeral global lists (all_processes, _isolation_audit) with embedded SQLite database (via SQLModel or SQLAlchemy).	Persists scan history, forensic snapshots, and tamper-resistant containment audit trails across server restarts.	Medium	Low
PHASE 4.1 Extension	**WSL2 / Docker Sandbox Integration Service** Implement SandboxService following the abstract provider interface. Orchestrate container detonation, snapshotting, and telemetry streaming.	Enables automated, isolated detonation of suspect samples without putting the host machine at risk.	Large	Medium
PHASE 4.2 Extension	**Software Supply Chain Security Scanner** Implement SupplyChainService to scan manifests (requirements.txt, package.json) and inspect PE import tables for suspicious unbacked imports.	Expands ShadowMem's defense envelope from pure in-memory runtime execution to proactive dependency vulnerability detection.	Large	Low

6. Architecture Comparison: Current Monolith vs. Target Modular Blueprint

The side-by-side comparison below illustrates the structural transformation from the current monolithic repository to the recommended enterprise architecture:

Current Monolithic Layout (Debt-Heavy)	Target Clean Architecture (Modular & Scalable)

Root Directory Monolith:
ShadowMem/
■■■ **app.py** (3,434 lines: API + Win32 + ML + HTML)
■■■ **server.py** (290 lines: Orphan mock on port 8000)
■■■ **memoryscanner.py** (185 lines: Win32 ctypes)
■■■ **detecter.py** (136 lines: CLI duplicator)
■■■ **technique_docs.py** (154 lines: Static dicts)
■■■ **train_model.py** (615 lines: ML pipeline)
■■■ **malware_simulator.py** (498 lines: Test harness)
■■■ **restart_server.py** (Taskkill port-killer)
■■■ **index.html** (Legacy standalone HTML)
■■■ **run.txt** / **launch_shadowmem.bat** (Port 8001/8000)
■■■ **frontend/** (Next.js app exporting to out/)

Flaws:
• No separation of concerns (HTML, Win32, routes entangled)
• Dual server port conflict (ERR-09)
• Unpersisted global state in memory
• No automated testing infrastructure

Structured Layered Package:
ShadowMem/
■■■ **main.py** (~60 lines: Application entry & lifecycle)
■■■ **config.py** (Pydantic-Settings & env variables)
■■■ **backend/**
■ ■■■ **api/** (API Layer)
■ ■ ■■■ **deps.py** (Dependency injection: auth, db, engine)
■ ■ ■■■ **v1/**
■ ■ ■■■ **router.py** (Aggregates routers)
■ ■ ■■■ **endpoints/** (scan, isolate, audit, ws)
■ ■ ■■■ **schemas/** (Pydantic DTOs)
■ ■■■ **services/** (Domain Logic)
■ ■ ■■■ **forensics_service.py** (Coordinates scan & ML)
■ ■ ■■■ **containment_service.py** (Process isolation)
■ ■ ■■■ **alert_broadcaster.py** (Live WebSocket hub)
■ ■ ■■■ **sandbox_service.py** (Docker / WSL2 interface)
■ ■ ■■■ **supply_chain_service.py** (Dependency scanner)
■ ■■■ **core/** (Low-Level Systems)
■ ■ ■■■ **win32/** (VirtualQueryEx, NtQuery wrappers)
■ ■ ■■■ **ml/** (Model loader, feature extraction, ONNX)
■ ■ ■■■ **security.py** (Auth tokens, CORS policies)
■ ■■■ **db/** (Persistence)
■ ■■■ **session.py** (SQLite engine)
■ ■■■ **models.py** (ScanHistory, AuditLog tables)
■■■ **frontend/** (Next.js static export)
■■■ **tests/** (Pytest suite with Win32 mocks)
■■■ **docs/** (OpenAPI specs & architecture guides)

7. Summary & Immediate Next Steps

ShadowMem possesses an exceptional defensive engineering core: its low-level memory traversal and thread-start heuristics accurately expose stealthy in-memory injection techniques (Reflective DLL, Process Hollowing, Shellcode beacons) that evade traditional file-based endpoint tools. By executing the phased upgrade roadmap outlined herein, the platform will transition from an ad-hoc hackathon prototype into an enterprise-grade, extensible cyber defense operations platform.

Recommended Immediate Action: *Proceed with* ***Phase 1.1 through Phase 1.3****. This permanently resolves the ERR-09 dual-server issue, stabilizes WebSocket alert broadcasting with live detection telemetry, introduces type-safe Pydantic schemas, and centralizes configuration—all without altering the core forensic algorithms or requiring a complete codebase rewrite.*